```
import { useState, useEffect, useRef, useCallback } from "react";

/* ── STYLES ──────────────────────────────────────────────────── */
const css = `
  @import url('https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800
  *{box-sizing:border-box;margin:0;padding:0}
  body,#root{font-family:'Syne',sans-serif;background:#07080f;color:#eeeeff;min-h
  .shell{max-width:720px;margin:0 auto;padding:36px 20px 80px}

  .hd{margin-bottom:32px}
  .hd-tag{font-family:'DM Mono',monospace;font-size:.72rem;color:#4040a0;letter-s
  .hd-title{font-size:2rem;font-weight:800;letter-spacing:-.03em;line-height:1.1;
  .hd-sub{font-size:.82rem;color:#444;margin-top:6px;font-family:'DM Mono',monosp

  .steps{display:flex;margin-bottom:28px;background:#0d0d1a;border:1px solid #1a1
  .step-btn{flex:1;padding:14px 8px;border:none;background:transparent;font-famil
  .step-btn:last-child{border-right:none}
  .step-btn.done{color:#4ecc96}
  .step-btn.active{background:#12122a;color:#fff}
  .step-num{width:22px;height:22px;border-radius:50%;background:#1a1a30;display:f
  .step-btn.active .step-num{background:#3030b0}
  .step-btn.done .step-num{background:#1a3a2a;color:#4ecc96}

  .card{background:#0d0d1a;border:1px solid #1a1a30;border-radius:20px;padding:28
  @keyframes fadeUp{from{opacity:0;transform:translateY(8px)}to{opacity:1;transfo
  .card-title{font-size:1.15rem;font-weight:800;letter-spacing:-.02em;margin-bott
  .card-desc{font-size:.78rem;color:#555;font-family:'DM Mono',monospace;margin-b

  label{display:block;font-size:.72rem;font-weight:700;color:#555;letter-spacing:
  textarea,input,select{width:100%;background:#080812;border:1px solid #1a1a30;bo
  textarea:focus,input:focus,select:focus{border-color:#3030a0}
  select option{background:#0d0d1a}
  .field{margin-bottom:16px}
  .row{display:flex;gap:12px}
  .row .field{flex:1}

  .btn{width:100%;padding:13px;border-radius:10px;border:none;background:linear-g
  .btn:hover:not(:disabled){opacity:.85;transform:translateY(-1px)}
  .btn:disabled{opacity:.35;cursor:not-allowed;transform:none}
  .btn-ghost{width:100%;padding:11px;border-radius:10px;border:1px solid #1a1a30;
  .btn-ghost:hover{border-color:#3030a0;color:#aaa}

  .loader{display:flex;gap:6px;align-items:center;padding:14px 0;color:#444;font-
  .dot{width:6px;height:6px;background:#4040c0;border-radius:50%;animation:bounce
```

```
.dot:nth-child(2){animation-delay:.2s}
.dot:nth-child(3){animation-delay:.4s}
@keyframes bounce{0%,80%,100%{transform:translateY(0);opacity:.3}40%{transform:

.job-chip{background:#0a0a1f;border:1px solid #1a1a35;border-radius:12px;paddin
.job-icon{font-size:1.6rem;line-height:1}
.job-company{font-size:.7rem;font-family:'DM Mono',monospace;color:#4040a0;text
.job-role{font-size:.95rem;font-weight:700}

.q-progress{font-family:'DM Mono',monospace;font-size:.72rem;color:#333;margin-
.q-bar-bg{height:3px;background:#1a1a30;border-radius:2px;margin-bottom:20px}
.q-bar-fill{height:3px;background:#3030d0;border-radius:2px;transition:width .4
.q-num{font-family:'DM Mono',monospace;font-size:.7rem;color:#3030d0;margin-bot
.q-text{font-size:1.05rem;font-weight:700;line-height:1.4;margin-bottom:16px;le

.voice-q-bar{display:flex;align-items:center;gap:10px;background:#080812;border
.speak-btn{flex-shrink:0;width:38px;height:38px;border-radius:50%;border:none;b
.speak-btn:hover{background:#22223a;color:#9090ff}
.speak-btn.speaking{background:#1a1a40;color:#9090ff;animation:pulseRing 1.5s i
@keyframes pulseRing{0%,100%{box-shadow:0 0 0 0 rgba(100,100,255,.4)}50%{box-sh
.speak-label{font-family:'DM Mono',monospace;font-size:.72rem;color:#444}
.speak-label.on{color:#6060cc}

.tip-box{background:#080812;border:1px solid #1a1a30;border-left:3px solid #303
.tip-box strong{color:#4040a0}

.mode-toggle{display:flex;gap:8px;margin-bottom:18px}
.mode-pill{flex:1;padding:10px;border-radius:10px;border:1px solid #1a1a30;back
.mode-pill.active{background:#12122a;border-color:#3030a0;color:#fff}
.mode-pill:hover:not(.active){color:#888}

.mic-area{background:#080812;border:1px solid #1a1a30;border-radius:14px;paddin
.mic-btn{width:72px;height:72px;border-radius:50%;border:none;background:#12122
.mic-btn.rec{background:#1a0a30;color:#c060ff;animation:pulseMic 1s infinite}
.mic-btn:hover:not(.rec){background:#1a1a3a;color:#8080ff}
@keyframes pulseMic{0%,100%{box-shadow:0 0 0 0 rgba(180,80,255,.5)}50%{box-shad
.mic-status{font-family:'DM Mono',monospace;font-size:.78rem;color:#444}
.mic-status.on{color:#c060ff}

.voice-wave{display:flex;align-items:center;justify-content:center;gap:3px;heig
.wave-bar{width:3px;border-radius:2px;background:#c060ff;animation:wave 1s infi
.wave-bar:nth-child(1){animation-delay:0s;height:8px}
.wave-bar:nth-child(2){animation-delay:.1s;height:16px}
.wave-bar:nth-child(3){animation-delay:.2s;height:22px}
.wave-bar:nth-child(4){animation-delay:.3s;height:14px}
.wave-bar:nth-child(5){animation-delay:.4s;height:20px}
.wave-bar:nth-child(6){animation-delay:.3s;height:10px}
```

```
  .wave-bar:nth-child(7){animation-delay:.2s;height:18px}
  @keyframes wave{0%,100%{transform:scaleY(.4);opacity:.5}50%{transform:scaleY(1)

  .transcript-box{margin-top:14px;text-align:left;background:#0d0d20;border:1px s
  .transcript-label{font-size:.68rem;color:#444;text-transform:uppercase;letter-s

  .score-row{display:flex;gap:10px;margin-bottom:18px}
  .score-pill{flex:1;background:#080812;border:1px solid #1a1a30;border-radius:10
  .score-n{font-size:1.5rem;font-weight:800;letter-spacing:-.03em}
  .score-n.g{color:#4ecc96}.score-n.y{color:#f0c060}.score-n.r{color:#f06060}
  .score-l{font-size:.68rem;color:#444;font-family:'DM Mono',monospace;margin-top

  .fb{background:#080812;border:1px solid #1a1a30;border-radius:12px;overflow:hid
  .fb-head{padding:10px 14px;background:#0d0d20;font-size:.72rem;font-weight:700;
  .fb-body{padding:14px;font-family:'DM Mono',monospace;font-size:.8rem;line-heig
  .fb-actions{display:flex;gap:6px}
  .copy-btn{background:#12122a;border:none;border-radius:6px;padding:3px 10px;col
  .copy-btn:hover{background:#1e1e40;color:#aaa}
  .li{display:flex;gap:8px;margin-bottom:6px;font-family:'DM Mono',monospace;font
  .li-dot{flex-shrink:0;margin-top:2px}

  .nav-btns{display:flex;gap:10px;margin-top:16px}
  .nav-btns .btn,.nav-btns .btn-ghost{flex:1;margin:0}

  .badge{display:inline-flex;background:#0d0d22;border:1px solid #1a1a35;border-r
  .badge:hover{background:#12122a;color:#9090ff}

  .summary-bar{display:flex;align-items:center;gap:12px;background:#080812;border
  .summary-ring{width:56px;height:56px;border-radius:50%;flex-shrink:0;display:fl
  .summary-ring.great{background:#0a2a1a;color:#4ecc96;border:2px solid #4ecc9640
  .summary-ring.ok{background:#2a2000;color:#f0c060;border:2px solid #f0c06040}
  .summary-ring.low{background:#2a0a0a;color:#f06060;border:2px solid #f0606040}
  .summary-label{font-size:.72rem;font-family:'DM Mono',monospace;color:#444;text
  .summary-val{font-size:1rem;font-weight:700}

  .err-box{background:#1a0808;border:1px solid #3a1010;border-radius:10px;padding
`;

/* ── API ──────────────────────────────────────────────────────── */
async function callClaude(system, user) {
  const res = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "claude-sonnet-4-20250514",
      max_tokens: 1000,
      system,
```

```
      messages: [{ role: "user", content: user }],
    }),
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const data = await res.json();
  return data.content?.[0]?.text || "";
}

/* ── TTS ──────────────────────────────────────────────────────── */
function useTTS() {
  const [speaking, setSpeaking] = useState(false);

  const speak = useCallback((text) => {
    if (!window.speechSynthesis) return;
    window.speechSynthesis.cancel();
    const utt = new SpeechSynthesisUtterance(text);
    utt.lang = "es-ES";
    utt.rate = 0.92;
    const voices = window.speechSynthesis.getVoices();
    const esVoice = voices.find(v => v.lang.startsWith("es"));
    if (esVoice) utt.voice = esVoice;
    utt.onstart = () => setSpeaking(true);
    utt.onend = () => setSpeaking(false);
    utt.onerror = () => setSpeaking(false);
    window.speechSynthesis.speak(utt);
  }, []);

  const stop = useCallback(() => {
    window.speechSynthesis?.cancel();
    setSpeaking(false);
  }, []);

  return { speak, stop, speaking };
}

/* ── STT ──────────────────────────────────────────────────────── */
function useSTT() {
  const [recording, setRecording] = useState(false);
  const supported = !!(window.SpeechRecognition || window.webkitSpeechRecognition
  const recRef = useRef(null);
  const cbRef = useRef(null);

  useEffect(() => {
    const SR = window.SpeechRecognition || window.webkitSpeechRecognition;
    if (!SR) return;
    const rec = new SR();
    rec.lang = "es-ES";
```

```
    rec.continuous = true;
    rec.interimResults = true;
    rec.onresult = (e) => {
      let final = "";
      for (let i = e.resultIndex; i < e.results.length; i++) {
        if (e.results[i].isFinal) final += e.results[i][0].transcript + " ";
      }
      if (final && cbRef.current) cbRef.current(final);
    };
    rec.onend = () => setRecording(false);
    rec.onerror = () => setRecording(false);
    recRef.current = rec;
  }, []);

  const startRec = useCallback((onChunk) => {
    cbRef.current = onChunk;
    try { recRef.current?.start(); setRecording(true); } catch (e) { console.warn
  }, []);

  const stopRec = useCallback(() => {
    try { recRef.current?.stop(); } catch (e) { console.warn(e); }
    setRecording(false);
  }, []);

  return { recording, supported, startRec, stopRec };
}

/* ── CONSTANTS ──────────────────────────────────────────────── */
const EXAMPLES = [
  { role: "Diseñador UX/UI", company: "Startup de tecnología", jd: "Buscamos dise
  { role: "Asistente Administrativo", company: "Empresa de consultoría", jd: "Asi
  { role: "Community Manager", company: "Agencia de marketing", jd: "Community ma
];

const ANGLES = [
  "situaciones pasadas con el método STAR (Situación-Tarea-Acción-Resultado)",
  "habilidades técnicas específicas del puesto",
  "alineación con la cultura y valores de la empresa",
  "resolución de conflictos interpersonales",
  "liderazgo e influencia sin autoridad formal",
  "manejo de deadlines y presión alta",
  "logros concretos medibles con métricas",
  "adaptación a cambios inesperados",
  "toma de decisiones con información limitada",
  "colaboración entre equipos o departamentos",
  "crecimiento profesional y aprendizaje continuo",
  "manejo de clientes difíciles o situaciones delicadas",
```

```
];

const SC = n => n >= 80 ? "g" : n >= 60 ? "y" : "r";
const AVG_LBL = avg =>
  avg >= 80 ? { label: "Excelente preparación ✓", cls: "great" }
  : avg >= 60 ? { label: "Buen nivel, sigue mejorando", cls: "ok" }
  : { label: "Necesitas más práctica", cls: "low" };

/* ── APP ──────────────────────────────────────────────────────── */
export default function App() {
  const [phase, setPhase] = useState("setup");

  // setup state
  const [jobRole, setJobRole] = useState("");
  const [company, setCompany]   = useState("");
  const [jdText, setJdText]     = useState("");
  const [numQ, setNumQ]         = useState("5");
  const [genLoading, setGenLoading] = useState(false);
  const [genErr, setGenErr]     = useState("");

  // practice state
  const [questions, setQuestions]   = useState([]);
  const [usedQs, setUsedQs]         = useState([]);
  const [currentQ, setCurrentQ]     = useState(0);
  const [answerMode, setAnswerMode] = useState("voice");
  const [answer, setAnswer]         = useState("");
  const [evalLoading, setEvalLoading] = useState(false);
  const [feedback, setFeedback]     = useState(null);
  const [fbErr, setFbErr]           = useState("");
  const [allResults, setAllResults] = useState([]);
  const [copied, setCopied]         = useState(null);

  const { speak, stop, speaking } = useTTS();
  const { recording, supported: micOk, startRec, stopRec } = useSTT();

  // auto-speak new question
  useEffect(() => {
    if (phase === "practice" && questions[currentQ]) {
      const t = setTimeout(() => speak(questions[currentQ].q), 500);
      return () => clearTimeout(t);
    }
  }, [phase, currentQ]); // eslint-disable-line

  /* ── GENERATE ── */
  const generateQuestions = async () => {
    if (!jobRole.trim() || !jdText.trim()) return;
    setGenLoading(true);
```

```
  setGenErr("");

  // random angles for this session
  const angles = [...ANGLES].sort(() => Math.random() - 0.5).slice(0, 4);
  const seed = Math.random().toString(36).slice(2, 8).toUpperCase();

  const avoidSection = usedQs.length
    ? `\n\nYA USADAS (NO repitas ni similares):\n${usedQs.map((q, i) => `${i +
    : "";

  const system =
    `Eres un experto en RRHH. Genera preguntas de entrevista ÚNICAS y ESPECÍFIC
    `NUNCA uses preguntas genéricas como "háblame de ti" o "cuál es tu fortalez
    `Responde ÚNICAMENTE con JSON válido sin texto extra ni markdown:\n` +
    `{"preguntas":[{"q":"pregunta","tip":"qué busca el entrevistador en una fra

  const user =
    `Puesto: ${jobRole}\nEmpresa: ${company || "no especificada"}\n` +
    `Descripción:\n${jdText}\n\n` +
    `Sesión ID: ${seed} | Genera exactamente ${numQ} preguntas frescas.\n` +
    `Ángulos para ESTA sesión: ${angles.join(" | ")}\n` +
    `Las preguntas deben mencionar detalles reales de la descripción del trabaj
    avoidSection;

  try {
    const raw = await callClaude(system, user);
    // Extract JSON object from anywhere in the response
    const match = raw.match(/\{[\s\S]*\}/);
    if (!match) throw new Error("No JSON in response");
    const parsed = JSON.parse(match[0]);
    const newQs = parsed.preguntas || [];
    if (!newQs.length) throw new Error("No se recibieron preguntas");
    setQuestions(newQs);
    setUsedQs(prev => [...prev, ...newQs.map(q => q.q)]);
    setPhase("practice");
    setCurrentQ(0); setAnswer(""); setFeedback(null); setFbErr(""); setAllResul
  } catch (e) {
    setGenErr("Error generando preguntas: " + e.message + ". Intenta de nuevo."
  }
  setGenLoading(false);
};

/* ── EVALUATE ── */
const evaluate = async () => {
  if (!answer.trim()) return;
  stop();
  setEvalLoading(true);
```

```
  setFbErr("");

  const system =
    `Eres coach de entrevistas. Evalúa la respuesta del candidato para el puest
    `Responde ÚNICAMENTE con JSON válido sin texto extra ni markdown:\n` +
    `{"claridad":85,"relevancia":70,"confianza":90,"resumen":"frase evaluando l

  const user =
    `Puesto: ${jobRole} en ${company}\nDescripción: ${jdText}\n` +
    `Pregunta: ${questions[currentQ].q}\nRespuesta: ${answer}`;

  try {
    const raw = await callClaude(system, user);
    const match = raw.match(/\{[\s\S]*\}/);
    if (!match) throw new Error("No JSON in response");
    const fb = JSON.parse(match[0]);
    setFeedback(fb);
    setAllResults(prev => [...prev, { q: questions[currentQ].q, answer, feedbac
  } catch (e) {
    setFbErr("No se pudo evaluar: " + e.message + ". Intenta de nuevo.");
  }
  setEvalLoading(false);
};

const nextQ = () => {
  stop(); stopRec();
  if (currentQ + 1 >= questions.length) setPhase("done");
  else { setCurrentQ(p => p + 1); setAnswer(""); setFeedback(null); setFbErr("
};

const restart = () => {
  stop(); stopRec();
  setPhase("setup");
  setJobRole(""); setCompany(""); setJdText("");
  setQuestions([]); setUsedQs([]);
  setCurrentQ(0); setAnswer(""); setFeedback(null); setFbErr(""); setAllResults
};

const toggleMic = () => {
  if (recording) { stopRec(); }
  else { setAnswer(""); startRec(chunk => setAnswer(prev => prev + chunk)); }
};

const copyText = (text, id) => {
  navigator.clipboard.writeText(text).catch(() => {});
  setCopied(id); setTimeout(() => setCopied(null), 2000);
};
```

```
const avgScore = allResults.length
  ? Math.round(allResults.reduce((a, r) => a + (r.feedback.claridad + r.feedbac
  : 0;

const pi = phase === "setup" ? 0 : phase === "practice" ? 1 : 2;

return (
  <>
    <style>{css}</style>
    <div className="shell">

      <div className="hd">
        <div className="hd-tag">// AI Interview Trainer · con voz</div>
        <div className="hd-title">Practica para el trabajo que quieres</div>
        <div className="hd-sub">Pega la descripción real → escucha las pregunta
      </div>

      <div className="steps">
        {[["Tu trabajo", 1], ["Practica", 2], ["Resultados", 3]].map(([lbl, n],
          <div key={n} className={`step-btn${i < pi ? " done" : i === pi ? " ac
            <div className="step-num">{i < pi ? "✓" : n}</div>
            {lbl}
          </div>
        ))}
      </div>

      {/* ── SETUP ── */}
      {phase === "setup" && (
        <div className="card" key="setup">
          <div className="card-title">¿A qué trabajo quieres aplicar?</div>
          <div className="card-desc">Pega la descripción real del anuncio. La A

          <div className="row">
            <div className="field">
              <label>Puesto</label>
              <input value={jobRole} onChange={e => setJobRole(e.target.value)}
            </div>
            <div className="field">
              <label>Empresa</label>
              <input value={company} onChange={e => setCompany(e.target.value)}
            </div>
          </div>

          <div className="field">
            <label>Descripción del trabajo</label>
            <textarea rows={6} placeholder="Pega aquí el texto del anuncio: req
```

```
    </div>

    <div style={{ marginBottom: 18 }}>
      <div style={{ fontSize: ".7rem", color: "#444", fontFamily: "'DM Mo
      {EXAMPLES.map(ex => (
        <span key={ex.role} className="badge" onClick={() => { setJobRole
          {ex.role}
        </span>
      ))}
    </div>

    <div className="field" style={{ maxWidth: 220 }}>
      <label>Número de preguntas</label>
      <select value={numQ} onChange={e => setNumQ(e.target.value)}>
        <option value="3">3 — Rápido</option>
        <option value="5">5 — Normal</option>
        <option value="7">7 — Completo</option>
        <option value="10">10 — Intensivo</option>
      </select>
    </div>

    <button className="btn" onClick={generateQuestions} disabled={genLoad
      {genLoading ? "Generando preguntas..." : "Empezar entrevista →"}
    </button>

    {genLoading && (
      <div className="loader">
        <div className="dot" /><div className="dot" /><div className="dot
        Creando preguntas para {company || "tu empresa"}...
      </div>
    )}
    {genErr && <div className="err-box">{genErr}</div>}
  </div>
)}

{/* ── PRACTICE ── */}
{phase === "practice" && questions[currentQ] && (
  <div className="card" key={`q${currentQ}`}>
    <div className="job-chip">
      <div className="job-icon">🏢</div>
      <div>
        <div className="job-company">{company || "Empresa"}</div>
        <div className="job-role">{jobRole}</div>
      </div>
    </div>

    <div className="q-progress">
```

```
  <span>Pregunta {currentQ + 1} de {questions.length}</span>
  <span style={{ color: "#3030a0" }}>{Math.round((currentQ / question
</div>
<div className="q-bar-bg">
  <div className="q-bar-fill" style={{ width: `${(currentQ / question
</div>

<div className="q-num">Pregunta {currentQ + 1}</div>
<div className="q-text">{questions[currentQ].q}</div>

<div className="voice-q-bar">
  <button className={`speak-btn${speaking ? " speaking" : ""}`} onCli
    {speaking ? "⏹" : "🔊"}
  </button>
  <span className={`speak-label${speaking ? " on" : ""}`}>
    {speaking ? "Leyendo pregunta..." : "Toca para escuchar la pregun
  </span>
</div>

{questions[currentQ].tip && (
  <div className="tip-box">
    <strong>💡 Qué busca el entrevistador:</strong> {questions[curren
  </div>
)}

{!feedback && (
  <>
    <div className="mode-toggle">
      <button className={`mode-pill${answerMode === "voice" ? " activ
        🎙️ Responder con voz
      </button>
      <button className={`mode-pill${answerMode === "text" ? " active
        ⌨️ Escribir respuesta
      </button>
    </div>

    {answerMode === "voice" && (
      <div className="mic-area">
        {recording && (
          <div className="voice-wave">
            {[1,2,3,4,5,6,7].map(i => <div key={i} className="wave-ba
          </div>
        )}
        <button className={`mic-btn${recording ? " rec" : ""}`} onCli
          {recording ? "⏹" : "🎙️"}
        </button>
        <div className={`mic-status${recording ? " on" : ""}`}>
```

```
            {!micOk
              ? "Tu navegador no soporta micrófono — usa el modo texto"
              : recording ? "Grabando... toca para detener"
              : answer ? "Grabación lista — puedes seguir o evaluar"
              : "Toca el micrófono y responde como en la entrevista rea
          </div>
          {answer && (
            <div className="transcript-box">
              <div className="transcript-label">Lo que dijiste:</div>
              {answer}
            </div>
          )}
        </div>
      )}

      {answerMode === "text" && (
        <div className="field">
          <label>Tu respuesta</label>
          <textarea rows={5} placeholder="Responde como si estuvieras e
        </div>
      )}

      <button className="btn" onClick={evaluate} disabled={evalLoading
        {evalLoading ? "Evaluando..." : "Ver feedback →"}
      </button>

      {evalLoading && (
        <div className="loader">
          <div className="dot" /><div className="dot" /><div className=
          Analizando tu respuesta...
        </div>
      )}
      {fbErr && <div className="err-box">{fbErr}</div>}
    </>
  )}

  {feedback && (
    <div style={{ animation: "fadeUp .3s ease" }}>
      <div className="score-row">
        {[["claridad","Claridad"],["relevancia","Relevancia"],["confian
          <div className="score-pill" key={k}>
            <div className={`score-n ${SC(feedback[k])}`}>{feedback[k]}
            <div className="score-l">{l}</div>
          </div>
        ))}
      </div>
```

```
          <div className="fb">
            <div className="fb-head">Diagnóstico</div>
            <div className="fb-body">
              {feedback.resumen && <div style={{ marginBottom: 12, color: "
              {feedback.fortalezas?.map((f, i) => (
                <div className="li" key={i}><span className="li-dot" style=
              ))}
              {feedback.mejoras?.map((m, i) => (
                <div className="li" key={i}><span className="li-dot" style=
              ))}
            </div>
          </div>

          <div className="fb">
            <div className="fb-head">
              <span>★ Respuesta ideal para {company}</span>
              <div className="fb-actions">
                <button className="copy-btn" onClick={() => speak(feedback.
                <button className="copy-btn" onClick={() => copyText(feedba
              </div>
            </div>
            <div className="fb-body">{feedback.version_ideal}</div>
          </div>

          <div className="nav-btns">
            {currentQ + 1 < questions.length
              ? <button className="btn" onClick={nextQ}>Siguiente pregunta
              : <button className="btn" onClick={() => setPhase("done")}>Ve
            }
          </div>
        </div>
      )}
    </div>
  )}

  {/* ── RESULTS ── */}
  {phase === "done" && (
    <div className="card" key="done">
      <div className="card-title">Entrevista completada 🎯</div>
      <div className="card-desc" style={{ marginBottom: 20 }}>
        Resultados para <strong style={{ color: "#eeeeff" }}>{jobRole}</str
      </div>

      {avgScore > 0 && (() => {
        const { label, cls } = AVG_LBL(avgScore);
        return (
          <div className="summary-bar">
```

```
                <div className={`summary-ring ${cls}`}>{avgScore}</div>
                <div><div className="summary-label">Puntaje promedio</div><div
              </div>
            );
          })()}

          {allResults.map((r, i) => (
            <div className="fb" key={i} style={{ marginBottom: 14 }}>
              <div className="fb-head">
                <span>P{i + 1}: {r.q.slice(0, 55)}{r.q.length > 55 ? "…" : ""}<
                <span style={{ color: "#4ecc96", fontWeight: 800 }}>
                  {Math.round((r.feedback.claridad + r.feedback.relevancia + r.
                </span>
              </div>
              <div className="fb-body">
                <div style={{ color: "#555", marginBottom: 8, fontStyle: "itali
                {r.feedback.mejoras?.map((m, j) => (
                  <div className="li" key={j}><span className="li-dot" style={{
                ))}
              </div>
            </div>
          ))}

          <button className="btn" onClick={generateQuestions} disabled={genLoad
            {genLoading ? "Generando..." : "Repetir con nuevas preguntas →"}
          </button>
          <button className="btn-ghost" onClick={restart}>Practicar para otro t
        </div>
      )}

    </div>
  </>
);
}
```